ASIGNATURA:

Teoría de la Computación

TEMA:

Práctica 6: Gramática Libre de contexto

Lenguaje propuesto: Romance

Octubre de 2026

Índice

1. Introducción 3
   1.1 Teoría de la computación 3
   1.2 Gramáticas en teoría de la computación 3
   1.3 Gramática libre de contexto 3
   1.4 Objetivo de la práctica 4
2. Marco teórico 5
   2.1 Jerarquía de Chomsky 5
   2.2 BNF (Backus-Naur Form) 6
3. Desarrollo 7
   3.1 Nombre del lenguaje propuesto: Romance 7
   3.2 Gramática BNF 8
      i. Programa 9
      ii. Sentencia 9
      iii. Declaración de variables 10
      iv. Lectura de variable 10
      v. Escritura de variables 11
      vi. Asignaciones 11
      vii. Expresiones aritméticas 11
      viii. Expresiones relacionales 12
      ix. Condición simple y extendida, con anidamiento 13
      x. Ciclo 13
   3.3 Programa de ejemplo 15
   3.4 Implementación del programa 16
      Evidencia de funcionamiento 19
   3.5 Árbol de análisis gramatical del programa de ejemplo 20
   3.6 Árbol sintáctico y recorrido en postorden 25
4. Conclusiones 28
5. Referencias 29

1. Introducción

1.1 Teoría de la computación

La teoría de la computación estudia qué problemas se pueden resolver con un procedimiento mecánico, con qué recursos y bajo qué modelo de máquina. Suele dividirse en tres ramas: la teoría de autómatas y lenguajes formales, que clasifica los lenguajes según el tipo de máquina capaz de reconocerlos; la teoría de la computabilidad, que pregunta si un problema tiene solución algorítmica; y la teoría de la complejidad, que pregunta cuánto cuesta resolverlo.

1.2 Gramáticas en teoría de la computación

Una gramática formal es un conjunto de reglas que describe cómo se generan las cadenas válidas de un lenguaje. Se define como una cuádrupla G = (T,V,S,R), donde T es el conjunto de símbolos terminales, V es el conjunto de símbolos no terminales R es el conjunto de reglas de producción y S el símbolo inicial. Partiendo de S y aplicando reglas de R se obtienen derivaciones y el lenguaje de gramática es el conjunto de cadenas formadas solo por terminales a las que se puede llegar así. Las gramáticas son la contraparte generativa de los autómatas: la gramática dice cómo construir las cadenas y el autómata cómo reconocerlas.

1.3 Gramática libre de contexto

Una gramática libre de contexto (GLC) es aquella en la que todas las producciones tienen la forma $A \longrightarrow \alpha$, con A un solo no terminal y α una cadena cualquiera de terminales y no terminales. Lo de «libre de contexto» significa que A se puede reemplazar sin importar qué símbolos tenga alrededor. Esa restricción es justo la que permite describir estructuras anidadas (paréntesis balanceados, bloques dentro de bloques, un if dentro de otro), cosa que las expresiones regulares no pueden hacer. Los lenguajes que generan son exactamente los que reconoce un autómata de pila, y por eso la sintaxis de prácticamente todos los lenguajes de programación se especifica con una GLC.

1.4 Objetivo de la práctica

El objetivo es diseñar la sintaxis de un lenguaje de programación propio, sin palabras reservadas en inglés, y especificarla completa con una gramática libre de contexto en notación BNF. El lenguaje debe cubrir declaraciones, asignaciones, expresiones aritméticas y relacionales, lectura, escritura, condicionales simples y extendidos con anidamiento, y un tipo de ciclo. Para comprobar que la gramática no se queda en el papel se implementó un intérprete en C que la sigue regla por regla, y con él se obtuvieron el árbol de análisis gramatical y el árbol sintáctico de un programa de ejemplo.

2. Marco teórico

2.1 Jerarquía de Chomsky

En 1956 Noam Chomsky propuso clasificar las gramáticas según las restricciones que imponen a sus producciones. Cada nivel contiene al siguiente: entre más restricciones, menos lenguajes se pueden describir, pero más sencilla es la máquina que los reconoce. En la tabla, A y B son no terminales, a es un terminal y α, β, γ son cadenas de terminales y no terminales.

Tabla 1. *Jerarquía de Chomsky.*

Tipo	Gramática	Forma de las producciones	Reconocedor	Ejemplo de lenguaje
0	Sin restricciones (recursivamente enumerable)	$\alpha \rightarrow \beta$, con α no vacía	Máquina de Turing	Programas que se detienen con cierta entrada
1	Sensible al contexto	$\alpha A \beta \rightarrow \alpha \gamma \beta$, con γ no vacía	Autómata linealmente acotado	$\{a^n b^n c^n \mid n \geq 1\}$
2	Libre de contexto	$A \rightarrow \gamma$	Autómata de pila	$\{a^n b^n \mid n \geq 0\}$, paréntesis balanceados
3	Regular	$A \rightarrow aB$ o $A \rightarrow a$	Autómata finito	Identificadores, números

En un compilador conviven dos de estos niveles. El analizador léxico trabaja con lenguajes regulares: identificadores, números y palabras reservadas se reconocen con autómatas finitos. El analizador sintáctico trabaja con un lenguaje libre de contexto. Y hay reglas que ni una GLC puede expresar, como «toda variable se declara antes de usarse» o «la etiqueta a la que se salta tiene que existir». Esas dependen del contexto y se dejan para el análisis semántico. En Romance esto aparece de forma muy concreta con el ciclo, como se verá en la sección 3.2.

2.2 BNF (Backus-Naur Form)

La forma de Backus-Naur es una notación para escribir gramáticas libres de contexto. John Backus la usó para describir ALGOL 58 y Peter Naur la ajustó para el reporte de ALGOL 60, el primer lenguaje cuya sintaxis se definió formalmente de principio a fin. Sus convenciones son pocas:

- Los no terminales van entre corchetes angulares: `<expresion>`.
- Los terminales se escriben sin comillas: `si`, `;`.
- `:=` se lee «se define como».
- `|` separa las alternativas de un mismo no terminal.
- ε representa la cadena vacía (en la BNF original simplemente se dejaba la alternativa en blanco).

La repetición se expresa con recursión. Una lista de una o más sentencias, por ejemplo, se escribe así:

```
<lista> := <sentencia> | <sentencia> <lista>
```

Existen extensiones (EBNF) que agregan llaves para repetición y corchetes para partes opcionales. Aquí se usa BNF pura para que cada producción corresponda directamente a un nodo del árbol de análisis gramatical.

3. Desarrollo

3.1 Nombre del lenguaje propuesto: Romance

El lenguaje se llama **Romance**. La idea es sencilla: cada tipo de instrucción toma sus palabras reservadas de una lengua romance actual distinta, así que un programa termina pareciendo una junta de la Unión Latina. La estructura, en cambio, es la de C: llaves para los bloques, punto y coma al final de cada sentencia, paréntesis alrededor de las condiciones, comentarios con `//` y los operadores relacionales de siempre.

Hay dos excepciones deliberadas a las costumbres de C. Las expresiones aritméticas se escriben en **notación polaca** (prefija) al estilo de LISP: cada operación va entre paréntesis, con el operador primero y luego sus dos operandos. Lo que en C sería `(n + 1) / 2.0` en Romance es `(/ (+ n 1) 2.0)`. Es lo único del lenguaje que viene de Polonia y no de la Romania, pero tiene una ventaja real: no hacen falta reglas de precedencia, porque cada paréntesis deja claro qué se calcula primero. La segunda excepción es el ciclo: en lugar de while o for, Romance tiene un **salto con etiqueta** (un goto), en francés.

Tabla 2. *Palabras reservadas de Romance por tipo de instrucción.*

Elemento	Lengua	Palabras	Traducción
Programa	Gallego	`comezo`	comienzo
Declaración	Portugués	`inteiro, real, cadeia`	entero, real, cadena
Asignación	Rumano	`fie`	sea («sea x igual a…»)
Lectura	Catalán	`llegeix`	lee
Escritura	Italiano	`scrivi`	escribe
Condicional	Español	`si, sino`	si, si no
Ciclo	Francés	`etiquette, allera, quand`	etiqueta, irá, cuando
Expr. aritméticas	Polaca, estilo LISP	`(+ a b)  -  *  /  %`	operador en prefijo
Expr. relacionales	Símbolos de C	`==  !=  <  >  <=  >=`	operador en infijo

Hay tres tipos de dato: `inteiro` para enteros, `real` para números con punto decimal y `cadeia` para texto. Con dos enteros, `/` hace división entera, como en C; si al menos un operando es real, la división es real. Un detalle curioso de la repartición: el francés no podía quedarse con el condicional porque su «si» choca con el del español, así que el ciclo usa `quand` («cuando») para su condición.

3.2 Gramática BNF

A continuación está la gramática completa, por partes. El símbolo inicial es `<programa>`. Antes conviene fijar el nivel léxico, es decir, cómo se forman los terminales que no son palabras fijas. Estas reglas son regulares (tipo 3); en la implementación las resuelve el analizador léxico y en los árboles aparecen ya como hojas. La gramatica de define como G donde: (T,V,S,R)

T = { comezo, inteiro, real, cadeia, fie, llegeix, scrivi, si, sino,

etiquette, allera, quand, +, -, *, /, %, ==, !=, <, >, <=, >=,

=, (,), {, }, ;, :, coma, punto, comilla doble,

a-z, A-Z, _, 0-9 }

V = { <programa>, <sentencias>, <sentencia>, <bloque>, <declaracion>,

<tipo>, <lista_decl>, <item_decl>, <lectura>, <escritura>,

<lista_esc>, <asignacion>, <expresion>, <expr_arit>, <op_arit>,

<operando>, <expr_rel>, <op_rel>, <condicional>, <resto_si>,

<etiqueta>, <salto>, <guarda>, <id>, <resto_id>, <numero>,

<digitos>, <digito>, <letra>, <cadena>, <caracteres>, <caracter> }

S = <programa>

R = las producciones de este apartado, de la i a la x y las reglas léxicas.

```
<id>          := <letra> <resto_id>
<resto_id>    := <letra> <resto_id> | <digito> <resto_id> | ε
<numero>      := <digitos> | <digitos> "." <digitos>
<digitos>     := <digito> | <digito> <digitos>
<cadena>      := “ <caracteres> "
<caracteres>  := <caracter> <caracteres> | ε
<letra>       := a | ... | z | A | ... | Z | _
<digito>      := 0 | 1 | ... | 9
<caracter>    := cualquier carácter excepto '"' y el salto de línea
```

Un identificador no puede coincidir con una palabra reservada. Los comentarios empiezan con `//` y llegan hasta el fin de la línea; el analizador léxico los descarta junto con los espacios.

i. Programa

```
<programa> := comezo <id> { <sentencias> }
```

Todo programa empieza con `comezo`, un nombre y un bloque entre llaves. No hay función principal ni encabezados: lo que está entre las llaves se ejecuta de arriba hacia abajo.

```
comezo Saludo { scrivi("hola"); }
```

ii. Sentencia

```
<sentencias> := <sentencia> <sentencias> | ε
<sentencia>  := <declaracion> | <asignacion> | <lectura> | <escritura>
              | <condicional> | <etiqueta> | <salto>
<bloque>     := { <sentencias> }
```

La lista de sentencias es recursiva por la derecha y admite el caso vacío, así que `{ }` es un bloque válido. `<bloque>` es la pieza que permite el anidamiento: todo bloque contiene `<sentencias>`, entre ellas puede haber otro condicional con su propio bloque, y así sin límite de profundidad. Cada alternativa de `<sentencia>` empieza con un terminal distinto

(`inteiro`/`real`/`cadeia`, `fie`, `llegeix`, `scrivi`, `si`, `etiquette`, `allera`), de modo que con ver el primer token ya se sabe qué regla aplicar. Eso es lo que hace posible el analizador descendente recursivo de la sección 3.4, que nunca tiene que retroceder.

iii. Declaración de variables

```
<declaracion> := <tipo> <lista_decl> ;
<tipo>        := inteiro | real | cadeia
<lista_decl>  := <item_decl> | <item_decl> , <lista_decl>
<item_decl>   := <id> | <id> = <expresion>
```

Como en C, se pueden declarar varias variables del mismo tipo en una línea, con o sin valor inicial. Las que no se inicializan arrancan en 0, 0.0 o cadena vacía según su tipo. Declarar dos veces el mismo nombre es un error.

```
inteiro n, i = 1, f = 1;
real m;
cadeia par = "y además es par";
```

iv. Lectura de variable

```
<lectura> := llegeix ( <id> ) ;
```

`llegeix` lee una línea de la entrada estándar y la convierte al tipo con el que se declaró la variable. Si el texto no se puede convertir (letras en un `inteiro`, por ejemplo), el intérprete reporta el error en lugar de guardar basura.

```
llegeix(n);
```

v. Escritura de variables

```
<escritura> := scrivi ( <lista_esc> ) ;
<lista_esc> := <expresion> | <expresion> , <lista_esc>
```

`scrivi` recibe una o más expresiones separadas por coma, las concatena sin separador y termina con un salto de línea. Se pueden mezclar cadenas, variables y expresiones aritméticas completas.

```
scrivi(n, “! = “, f);
scrivi("el doble es ", (* 2 n));
```

vi. Asignaciones

```
<asignacion> := fie <id> = <expresion> ;
<expresion>  := <expr_arit> | <cadena>
```

La asignación empieza con `fie`, el subjuntivo rumano de «ser»: `fie x = 5` se lee más o menos «sea x igual a 5», como el *sea* de los enunciados matemáticos. A diferencia de C, la palabra clave deja ver de un vistazo qué líneas cambian el estado del programa. Si la variable es `inteiro` y el valor es real, se trunca; asignar una cadena a una variable numérica (o al revés) es un error de tipos.

```
fie f = (* f i);
fie i = (+ i 1);
```

vii. Expresiones aritméticas

```
<expr_arit> := ( <op_arit> <expr_arit> <expr_arit> ) | <operando>
<op_arit>   := + | - | * | / | %
<operando>  := <id> | <numero>
```

Es la parte más corta de la gramática, y es gracias a la notación polaca. En infijo, para respetar la precedencia sin ambigüedad, se necesitan al menos tres niveles (expresión, término y factor) más una regla para los paréntesis. En prefijo basta una producción recursiva: un paréntesis, un operador, exactamente dos expresiones y el paréntesis que cierra. Cada operación queda delimitada por sus propios paréntesis, así que la cadena se lee de izquierda a derecha sin retroceder y solo admite un árbol de derivación; esta parte de la gramática no es ambigua. Igual que en LISP, los paréntesis son obligatorios: cada par corresponde a exactamente una operación.

Tabla 3. *Equivalencias entre notación infija (C) y polaca estilo LISP (Romance).*

Infijo (C)	Romance
`a + b`	`(+ a b)`
`(n + 1) / 2.0`	`(/ (+ n 1) 2.0)`
`a * b + c * d`	`(+ (* a b) (* c d))`
`a - (b - c)`	`(- a (- b c))`
`(a - b) - c`	`(- (- a b) c)`
`n % 2`	`(% n 2)`

No hay menos unario; para obtener −x se escribe `(- 0 x)`.

viii. Expresiones relacionales

```
<expr_rel> := <expr_arit> <op_rel> <expr_arit>
<op_rel>   := == | != | < | > | <= | >=
```

Las comparaciones sí van en infijo, al estilo C. Mezclar las dos notaciones funciona bien: cada lado de la comparación es una expresión aritmética completa, cerrada por sus propios paréntesis, y el operador relacional queda en medio como una bisagra visible entre dos cálculos.

```
(% n 2) == 0          // n % 2 == 0
(+ i 1) <= (* 2 n)    // i + 1 <= 2 * n
```

Las expresiones relacionales solo aparecen como condición de `si` y de `allera ... quand`; no se pueden guardar en variables, porque Romance no tiene tipo lógico.

ix. Condición simple y extendida, con anidamiento

```
<condicional> := si ( <expr_rel> ) <bloque> <resto_si>
<resto_si>    := sino <bloque> | sino <condicional> | ε
```

Con `<resto_si>` ⟶ ε se obtiene el if-then simple; con `sino <bloque>`, el if-then-else. La tercera alternativa permite encadenar `sino si (...) { }` sin acumular llaves. Las llaves son obligatorias, y eso resuelve de raíz el problema clásico del *else colgante*: en C, `if (a) if (b) x; else y;` tiene dos árboles posibles porque no se sabe a qué if pertenece el else. En Romance cada `sino` va justo después de la llave que cierra el bloque de su `si`, así que hay una sola forma de derivarlo. El anidamiento sale de `<bloque>`, que contiene `<sentencias>`, que a su vez puede contener otro `<condicional>`.

```
si (n < 0) {
    scrivi("negativo");
} sino si (n == 0) {
    scrivi("cero");
} sino {
    si ((% n 2) == 0) {
        scrivi("positivo y par");
    }
}
```

x. Ciclo

```
<etiqueta> := etiquette <id> :
<salto>    := allera <id> <guarda> ;
<guarda>   := quand ( <expr_rel> ) | ε
```

El único ciclo de Romance es el goto, en francés. `etiquette` marca una posición dentro de un bloque y `allera` («irá», futuro de *aller*) salta a ella, ya sea de forma incondicional o solo cuando (`quand`) se cumple una condición. Con un salto condicional hacia atrás se arma un ciclo con prueba al final, que es exactamente un do-while:

Tabla 4. *El ciclo de Romance frente al do-while de C.*

C (do-while)	Romance

`do {`
`  f = f * i;`
`  i = i + 1;`
`} while (i <= n);`
	`etiquette ciclo:`
	`fie f = (* f i);`
	`fie i = (+ i 1);`
	`allera ciclo quand (i <= n);`

Y con un salto hacia adelante al principio se obtiene la versión con prueba al inicio, equivalente a un while:

```
etiquette arriba:
si (i > n) { allera fuera; }
fie s = (+ s i);
fie i = (+ i 1);
allera arriba;
etiquette fuera:
```

Aquí aparece un detalle interesante desde el punto de vista teórico. La gramática acepta `allera fin;` aunque no exista ninguna `etiquette fin:` en el programa. Comprobar que el nombre del salto coincide con el de una etiqueta es una restricción dependiente del contexto, de la misma familia que el lenguaje {wcw}, el ejemplo clásico de lenguaje que no es libre de contexto y que se usa justamente para modelar «declarar antes de usar» (Aho et al., 2008). Por eso esa verificación no está en la BNF sino en el intérprete, junto con la regla de alcance: se puede saltar a una etiqueta del mismo bloque o de un bloque que lo contenga, pero nunca hacia dentro de un bloque. Dijkstra (1968) tenía sus razones para desconfiar del goto; con estas reglas, al menos, nadie puede aterrizar a la mitad de un `si`.

3.3 Programa de ejemplo

El siguiente programa usa todas las instrucciones del lenguaje: los tres tipos de declaración (con y sin valor inicial), lectura, escritura de cadenas, variables y expresiones, asignaciones con expresiones aritméticas anidadas, un condicional extendido con un ciclo dentro y, dentro de esa misma rama, un condicional simple anidado. Lee un entero n; si es negativo lo rechaza, y si no, calcula n! con el ciclo goto, saca la media de los enteros de 1 a n y avisa si n es par.

```
// Factorial, media y paridad de un entero
comezo Ejemplo {
    inteiro n, i = 1, f = 1;
    real m;
    cadeia par = "y además es par";

    scrivi("Dame un entero:");
    llegeix(n);

    si (n < 0) {
        scrivi("No hay factorial para negativos");
    } sino {
        etiquette ciclo:
        fie f = (* f i);
        fie i = (+ i 1);
        allera ciclo quand (i <= n);

        fie m = (/ (+ n 1) 2.0);
        scrivi(n, "! = ", f, ", media de 1 a n = ", m);
        si ((% n 2) == 0) {
            scrivi(par);
        }
    }
}
```

Como el ciclo prueba la condición al final, el cuerpo se ejecuta al menos una vez. Con n = 0 eso da f = 1 · 1 = 1, que coincide con 0! = 1, así que ese caso no necesita tratamiento aparte.

3.4 Implementación del programa

Para probar la gramática se escribió `romance.c`, un intérprete en C (estándar C99) que solo usa `stdio.h`. Trabaja en dos etapas:

1. **Analizador léxico.** Lee el archivo completo, lo recorre carácter por carácter y llena un arreglo de tokens (palabra reservada, identificador, número, cadena o símbolo), descartando espacios y comentarios.

2. **Analizador sintáctico descendente recursivo que ejecuta conforme lee.** Hay una función por cada no terminal de la BNF; cada una consume los terminales de su producción y llama, en el mismo orden, a las funciones de los no terminales que aparecen en ella. Todas reciben un parámetro `ejecutar`: si vale 1, la función lee su parte del programa y además la ejecuta; si vale 0, solo la lee. Así, la rama de un `si` cuya condición es falsa se recorre igual (para validar su sintaxis), pero sin ejecutarse.

Las variables se guardan en un arreglo de estructuras con nombre, tipo y valor. Como no se usa `string.h` ni `stdlib.h`, las pocas utilidades de texto que hacen falta (comparar, copiar y medir cadenas) están escritas a mano, y las conversiones de texto a número se hacen con `sscanf`, que sí pertenece a `stdio.h`.

La correspondencia entre gramática y código es casi literal. Esta es la producción de las expresiones aritméticas:

```
/* <expr_arit> := "(" <op_arit> <expr_arit> <expr_arit> ")" | <operando> */
Valor expr_arit(int ejecutar)
{
    if (ve("(")) {
        char op[3];
        int linea = tokens[pos].linea;
        Valor a, b;

        come("(");
        if (!es_operador_aritmetico())
            error_sintaxis("un operador aritmetico");
        copia(op, tokens[pos].texto, 2);
        pos++;
        a = expr_arit(ejecutar);
        b = expr_arit(ejecutar);
        come(")");
        if (ejecutar && !hay_error)
            return opera(op, a, b, linea);
        return a;
    }
    return operando(ejecutar);
}
```

Y esta la del condicional, donde se ve cómo el parámetro `ejecutar` decide qué rama corre:

```
void condicional(int ejecutar)
{
    int cumple;

    come("si");
    come("(");
    cumple = expr_rel(ejecutar);
    come(")");
    bloque(ejecutar && cumple);
    if (ve("sino")) {
        come("sino");
        if (ve("si"))
            condicional(ejecutar && !cumple);
        else
            bloque(ejecutar && !cumple);
    }
}
```

El salto se resuelve en `sentencias`. Cuando un `allera` se dispara, deja anotado el nombre de la etiqueta. La lista de sentencias busca esa etiqueta en su propio bloque, sin entrar a bloques internos; si la encuentra, mueve la posición de lectura hasta ella y sigue ejecutando desde ahí. Si no, termina de leer el bloque sin ejecutarlo y el bloque de afuera repite la búsqueda. Si se llega al final del programa sin encontrarla, se reporta el error.

```
void sentencias(int ejecutar)
{
    int inicio = pos;

    while (!fin_de_bloque()) {
        sentencia(ejecutar);
        if (saltando && ejecutar) {
            int destino = busca_etiqueta(inicio, etiqueta_destino);
            if (destino >= 0) {
                pos = destino;
                saltando = 0;
            } else {
                ejecutar = 0;
            }
        }
    }
}
```

Se compila con `gcc -std=c99 -o romance romance.c` y se ejecuta con `./romance archivo.rom`.

Evidencia de funcionamiento

Se compiló el intérprete y se corrió el programa de ejemplo con tres entradas: un número par, uno impar y uno negativo. La entrada se pasó con `echo` por tubería. Esta es la salida de la terminal:

```
$ gcc -std=c99 -Wall -o romance romance.c

$ echo 6 | ./romance ejemplo.rom
Dame un entero:
6! = 720, media de 1 a n = 3.5
y además es par

$ echo 5 | ./romance ejemplo.rom
Dame un entero:
5! = 120, media de 1 a n = 3

$ echo -3 | ./romance ejemplo.rom
Dame un entero:
No hay factorial para negativos
```

Con 6 se recorre la rama `sino` completa: el ciclo calcula 720 = 6!, la media de 1 a 6 es 3.5 y, como 6 es par, entra al `si` anidado. Con 5 la media es exactamente 3 y no aparece el mensaje de paridad. Con −3 se toma la rama del `si`.

3.5 Árbol de análisis gramatical del programa de ejemplo

El árbol completo del programa de ejemplo tiene 311 nodos, demasiados para una página legible, así que se presenta en partes. La Figura 1 muestra la raíz y la cadena de `<sentencias>` del nivel superior. Los nodos con borde punteado y el símbolo ▲ son subárboles que se desarrollan completos en las figuras siguientes. Los no terminales van en recuadro y los terminales (las hojas) en negritas; leyendo las hojas de todas las figuras de izquierda a derecha se reconstruye el programa token por token.

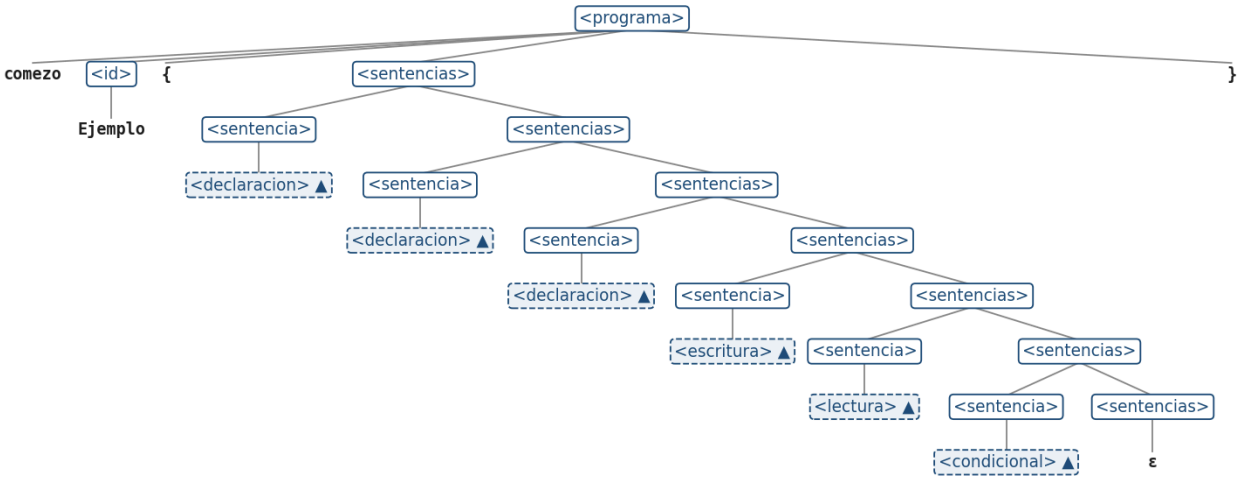


Figura 1. *Raíz del árbol: <programa> y las seis sentencias del nivel superior.*

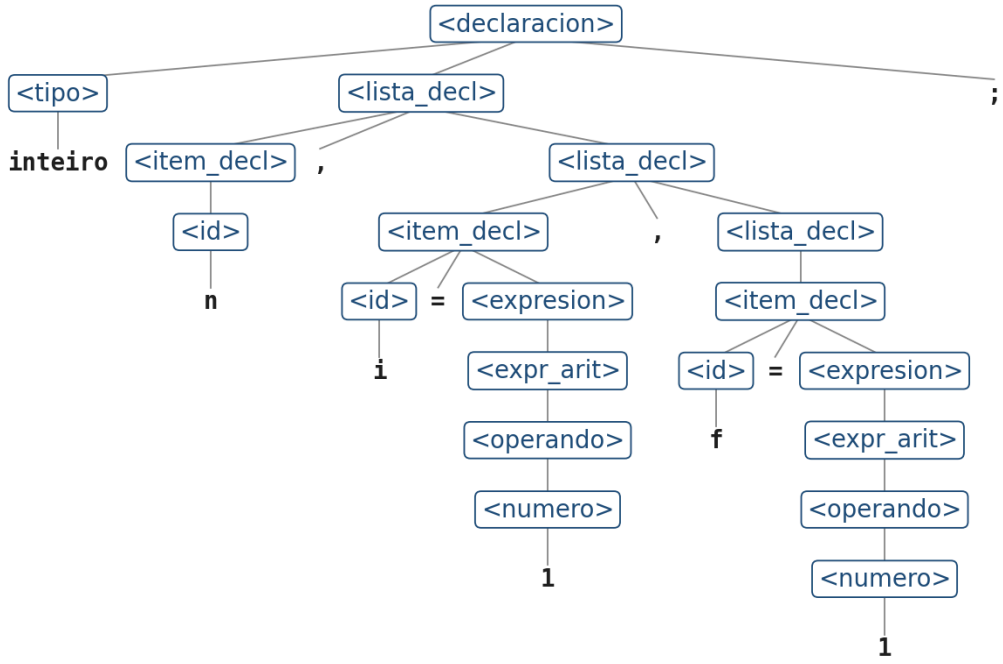


Figura 2. *Subárbol de inteiro n, i = 1, f = 1;*

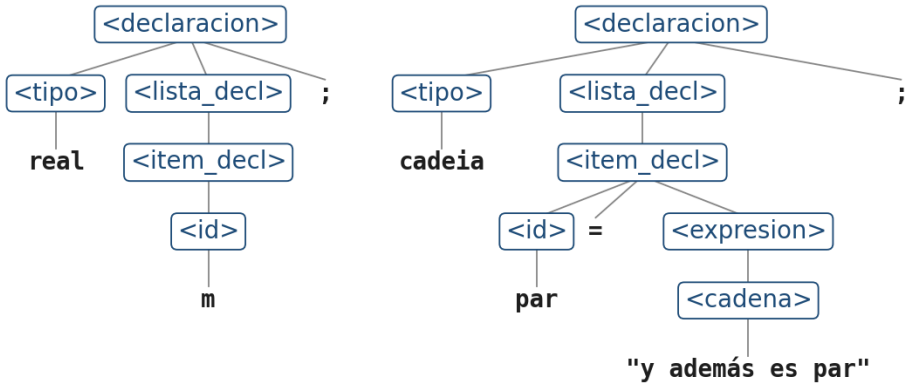


Figura 3. *Subárboles de real m; y cadeia par = "y además es par";*

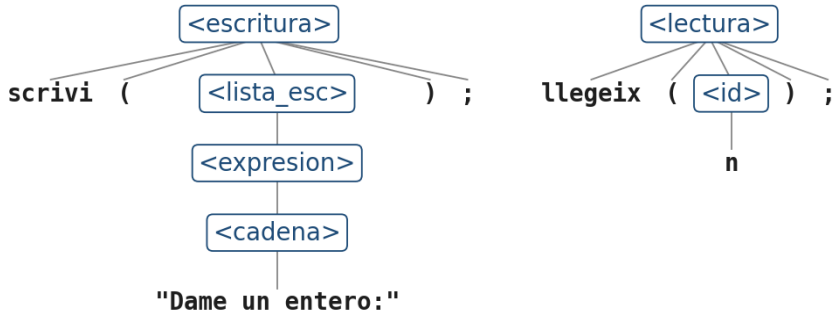


Figura 4. *Subárboles de scrivi("Dame un entero:"); y llegeix(n);*

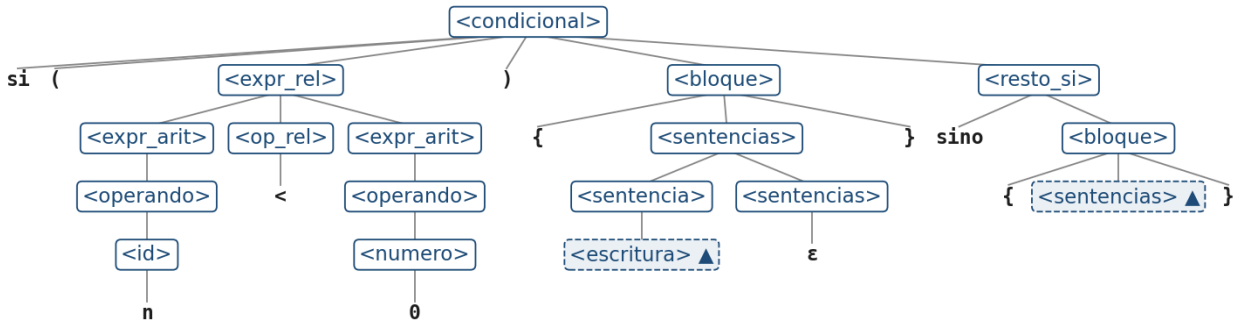


Figura 5. *Condicional extendido externo, si (n < 0) { … } sino { … }. La lista de sentencias del sino se desarrolla en la Figura 7.*

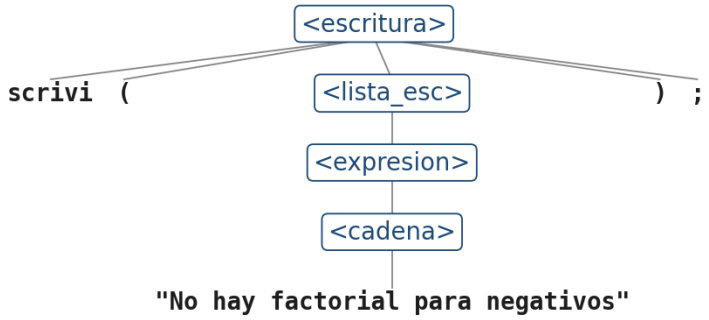


Figura 6. *Escritura de la rama si: scrivi("No hay factorial para negativos");*

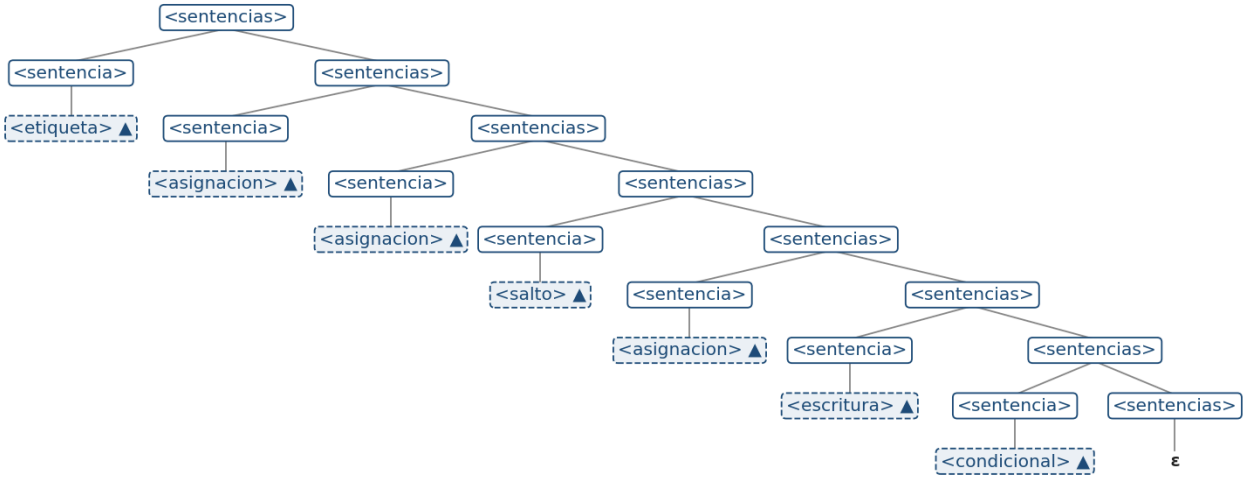


Figura 7. *Lista de sentencias del bloque sino.*

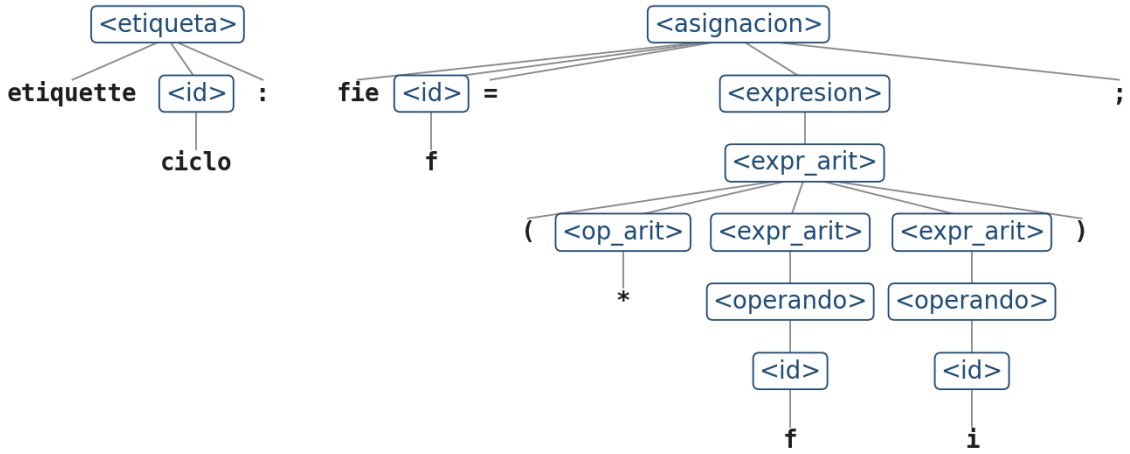


Figura 8. *Subárboles de etiquette ciclo: y fie f = (* f i);*

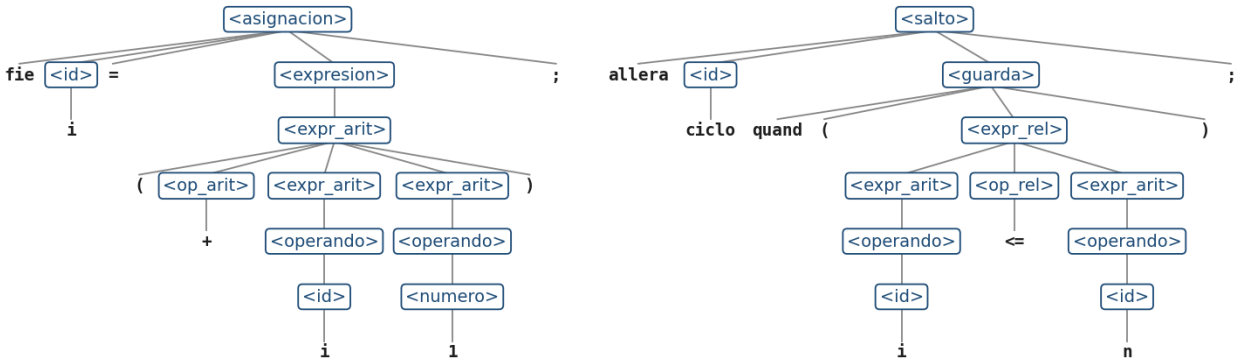


Figura 9. *Subárboles de fie i = (+ i 1); y allera ciclo quand (i <= n);*

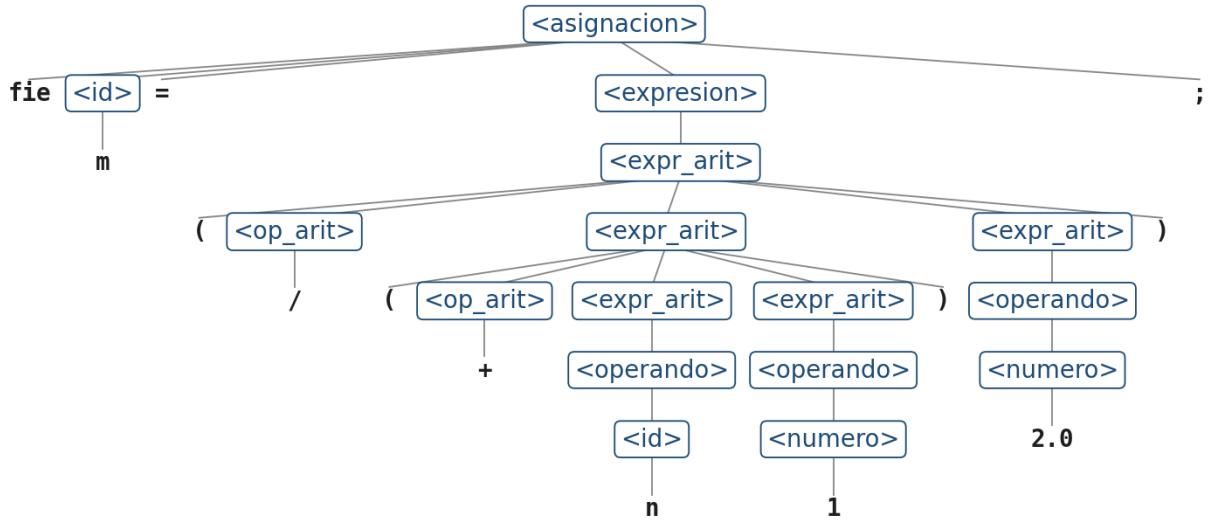


Figura 10. *Subárbol de fie m = (/ (+ n 1) 2.0);*

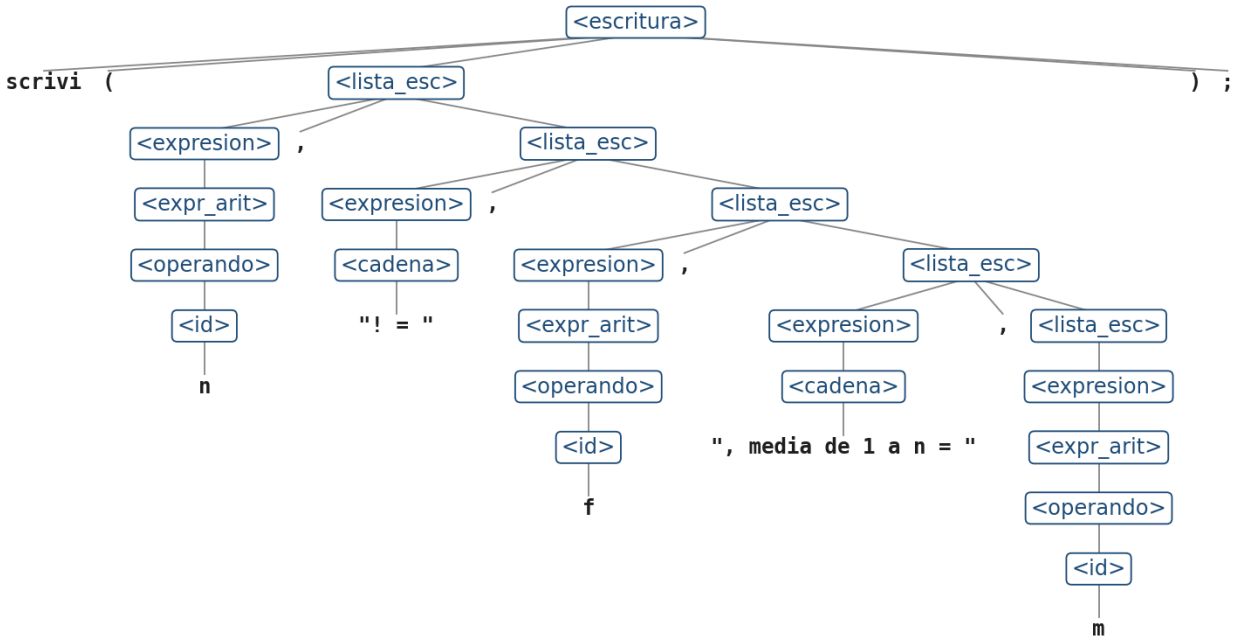


Figura 11. *Subárbol de scrivi(n, "! = ", f, ", media de 1 a n = ", m);*

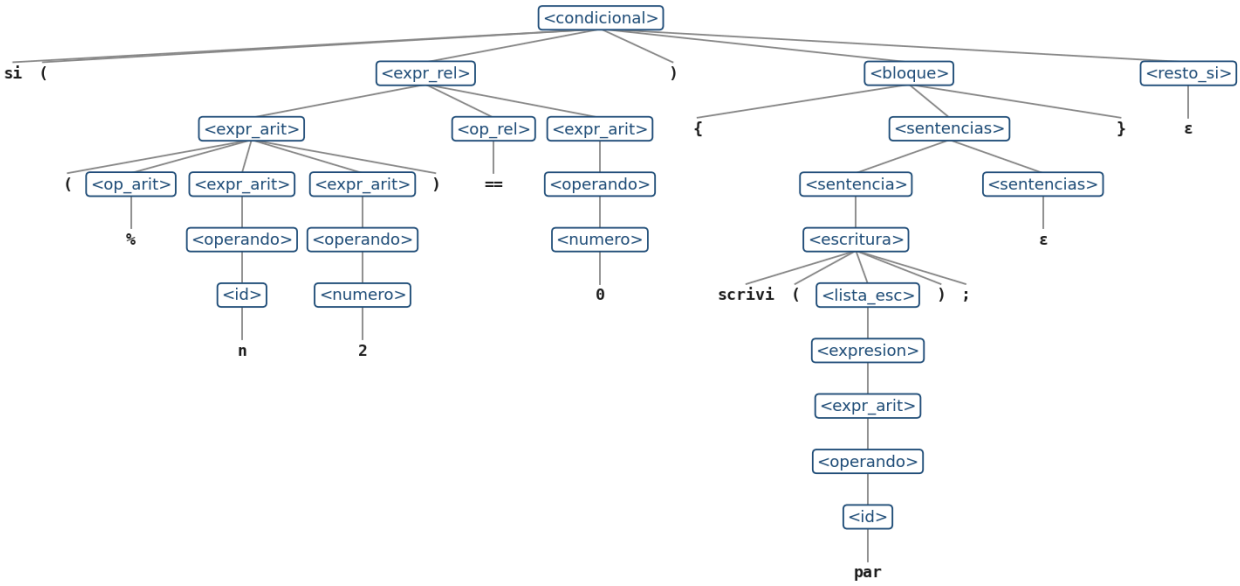


Figura 12. *Condicional simple anidado, si ((% n 2) == 0) { scrivi(par); }*

3.6 Árbol sintáctico y recorrido en postorden

El árbol sintáctico abstracto se queda solo con la estructura que importa para ejecutar. Los operadores son nodos internos y sus operandos son hijos, así que `(/ (+ n 1) 2.0)` se convierte en

un nodo `/` cuyo hijo izquierdo es el subárbol de `+`. Las sentencias se representan con el nombre de la instrucción (`decl inteiro`, `scrivi`, `llegeix`, `si`, `allera ciclo`…) y el condicional tiene como hijos la condición, un nodo `entonces` y un nodo `sino`. Un dato curioso: si a una expresión de Romance se le quitan los paréntesis, lo que queda es literalmente el recorrido en preorden de su árbol, y cada par de paréntesis marca dónde empieza y termina un subárbol. Es la misma razón por la que en LISP el código se parece tanto a su propio árbol.

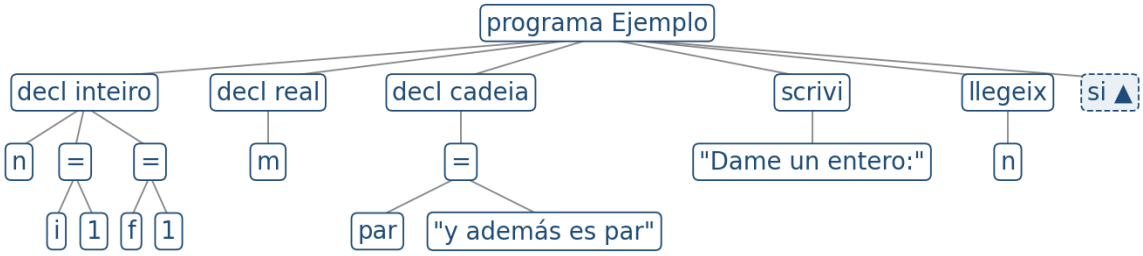


Figura 13. *Árbol sintáctico, nivel superior. El nodo si se desarrolla en las Figuras 14 y 15.*

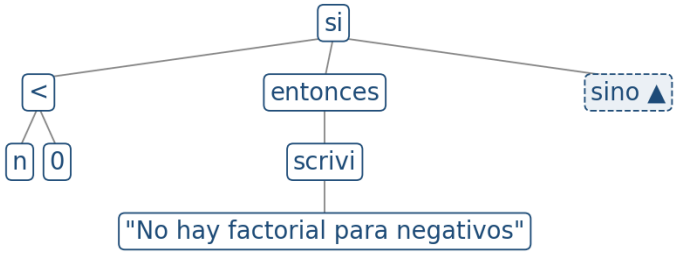


Figura 14. *Subárbol del condicional externo; el nodo sino se desarrolla en la Figura 15.*

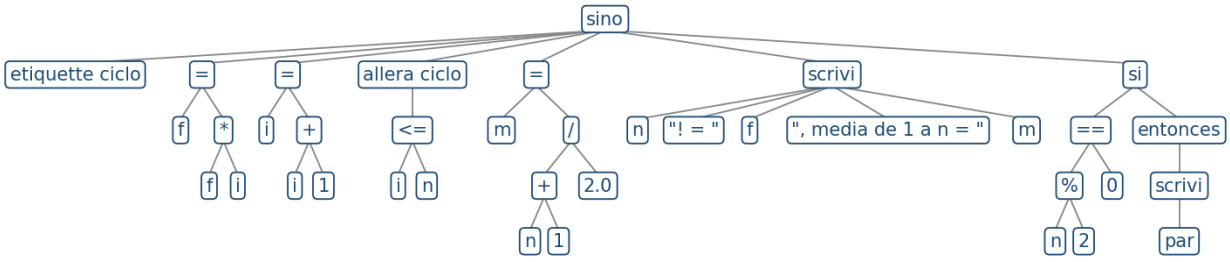


Figura 15. *Subárbol del bloque sino: ciclo, cálculo de la media, escritura y condicional anidado.*

El recorrido en postorden visita primero los hijos, de izquierda a derecha, y al final el nodo. Para el programa de ejemplo queda la siguiente secuencia (los nodos se separan con ·):

```
n · i · 1 · = · f · 1 · = · decl inteiro · m · decl real · par ·
"y además es par" · = · decl cadeia · "Dame un entero:" · scrivi · n · llegeix ·
n · 0 · < · "No hay factorial para negativos" · scrivi · entonces ·
etiquette ciclo · f · f · i · * · = · i · i · 1 · + · = · i · n · <= ·
allera ciclo · m · n · 1 · + · 2.0 · / · = · n · "! = " · f ·
", media de 1 a n = " · m · scrivi · n · 2 · % · 0 · == · par · scrivi ·
entonces · si · sino · si · programa Ejemplo
```

En las expresiones, el postorden da la notación polaca inversa: para `(/ (+ n 1) 2.0)` el recorrido produce `n 1 + 2.0 /`, que es el orden en que una máquina de pila evaluaría la expresión (apila n, apila 1, suma, apila 2.0, divide). Es la misma idea que usan muchos compiladores para generar código intermedio a partir del árbol.

4. Conclusiones

Lo más útil de la práctica fue ver que cada decisión de diseño en la gramática tiene consecuencias inmediatas en el analizador. Hacer que cada sentencia empiece con una palabra reservada distinta permitió escribir el analizador descendente recursivo casi como una traducción directa de la BNF, sin retrocesos ni trucos. Con una gramática mal pensada, el mismo analizador habría necesitado más tokens de anticipación o reescribir reglas para quitar ambigüedades. Escribirlo en C con solo `stdio.h` obligó además a mantener el intérprete simple: lee y ejecuta en una sola pasada, sin construir estructuras intermedias.

La notación polaca resultó ser un buen intercambio, aunque no gratis. Por el lado de la gramática, redujo las expresiones aritméticas a una sola producción recursiva, sin reglas de precedencia y sin ambigüedad. Por el lado humano, los paréntesis al estilo LISP fueron necesarios: sin ellos, algo como `+ * a b * c d` obliga a llevar una pila mental para entenderlo, mientras que `(+ (* a b) (* c d))` se lee de un vistazo, a cambio de escribir más paréntesis. Las llaves obligatorias en el `si` tienen un costo parecido, un poco más de texto, a cambio de eliminar por completo el else colgante.

El ciclo goto dejó a la vista el límite de las gramáticas libres de contexto. La gramática puede decir que después de `allera` va un identificador, pero no puede exigir que ese identificador sea el de una etiqueta existente. Esa separación entre lo que valida la sintaxis y lo que tiene que validar el análisis semántico es la misma que hacen los compiladores reales con las declaraciones de variables, y aquí salió de forma natural en lugar de quedarse como un dato de la teoría.

Por último, mezclar seis idiomas en un lenguaje fue más divertido que problemático. Lo único que hubo que cuidar fueron los choques, como el «si» que comparten el español y el francés, y que las palabras elegidas no se confundieran con identificadores comunes.

5. Referencias

Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.ª ed.). Pearson Educación.

Backus, J. W., Bauer, F. L., Green, J., Katz, C., McCarthy, J., Naur, P. (Ed.), Perlis, A. J., Rutishauser, H., Samelson, K., Vauquois, B., Wegstein, J. H., van Wijngaarden, A. y Woodger, M. (1963). Revised report on the algorithmic language ALGOL 60. *Communications of the ACM, 6*(1), 1–17.

Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory, 2*(3), 113–124.

Dijkstra, E. W. (1968). Go to statement considered harmful. *Communications of the ACM, 11*(3), 147–148.

Hopcroft, J. E., Motwani, R. y Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3.ª ed.). Pearson Educación.

Sipser, M. (2013). *Introduction to the theory of computation* (3rd ed.). Cengage Learning.